Patch du framework

Pour faciliter les tests des versions avec ou sans JavaScript, modifiez le framework de la façon décrite ci-dessous.

Ainsi, vous pourrez changer la valeur du paramètre `disable_js` pour activer ou désactiver JavaScript tout en autorisant le JavaScript de Bootstrap.

Adaptez les chemins de `enabled_paths` pour qu'ils correspondent aux chemins utilisés dans votre projet.

config/dev.ini

```
...
[JavaScript]
disable_js = false
; les chemins actifs malgré la désactivation de JS
; Chemins relatifs locaux
;enabled_paths[] = "css/bootstrap-5.3.3-dist/js/bootstrap.min.js"
;enabled_paths[] = "lib/popper.min.js"
; URLs CDN
enabled_paths[] =
"https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.min.js"
enabled_paths[] =
"https://cdnjs.cloudflare.com/ajax/libs/popper.js/2.11.8/umd/popper.min.js"
...
```

Remplacez le contenu du fichier `framework/Router.php` par celui-ci :

framework/Router.php

```php
<?php

require_once 'Controller.php';
require_once 'Configuration.php';
require_once 'Tools.php';

class Router {

    private function sanitize_all_array(array $array): array {
        $copy = [];
        foreach ($array as $key => $value) {
            if (is_array($value)) {
                $copy[$key] = $this->sanitize_all_array($value);
            } else {
                if (!Configuration::get("insafe_inputs") ||
(Configuration::get("insafe_inputs") && !in_array($key,
Configuration::get("insafe_inputs"))))
```

```
                    $copy[$key] = Tools::sanitize($value);
                else
                    $copy[$key] = $value;
            }
        }
        return $copy;
    }

    private function sanitize_all_input(): void {
        $_GET = $this->sanitize_all_array($_GET);
        $_POST = $this->sanitize_all_array($_POST);
        $_REQUEST = $this->sanitize_all_array($_REQUEST);
    }

    //sur base de la requête, renvoie une instance du controlleur demandé.
    private function get_controller(): Controller {
        $controller_name = Configuration::get("default_controller");
        if (isset($_GET['controller']) && $_GET['controller'] != "") {
            $controller_name = $_GET['controller'];
        }

        $parts = preg_split('/_+/', $controller_name);
        $parts = array_map(function($p) { return ucfirst(strtolower($p)); },
$parts);
        $controller_class_name = 'Controller' . implode('', $parts);

        $filename = "controller/$controller_class_name.php";

        if (file_exists($filename)) {
            require_once "controller/$controller_class_name.php";
            if (class_exists($controller_class_name)) {
                return new $controller_class_name();
            }
        }
        throw new Exception("Controller '$controller_name' does'nt exist");

    }

    //sur base de la requête, appelle l'action (méthode) sur le controlleur
donné.
    private function call_action(Controller $controller): void {
        $action_name = "index";
        if (isset($_GET['action']) && $_GET['action'] != "") {
            $action_name = strtolower($_GET['action']);
        }

        if (method_exists($controller, $action_name)) {
            $controller->$action_name();
        } else {
            throw new Exception("Action '$action_name' does'nt exist in this
controller.");
        }
    }

    /*
     * Désactive JS si le flag 'disable_js' est à true dans dev.ini,
     * à l'exception des paths repris dans 'enabled_paths'.
```

```
     */
    private function check_disable_js(): void {
        if (Configuration::get("disable_js", false)) {
            $protocol = isset($_SERVER['HTTPS']) && $_SERVER['HTTPS'] === 'on'
? 'https' : 'http';
            $host = $_SERVER['HTTP_HOST'];
            $web_root = Configuration::get("web_root");
            $baseUrl = $protocol . '://' . $host . $web_root;
            $enabled_paths_config = Configuration::get("enabled_paths",
false);

            $paths_array = [];
            if ($enabled_paths_config) {
                if (is_array($enabled_paths_config)) {
                    $paths_array = $enabled_paths_config;
                } else {
                    $enabled_paths = trim($enabled_paths_config);
                    if ($enabled_paths) {
                        $paths_array = preg_split("/[\s;:]+/",
$enabled_paths);
                    }
                }
            }

            if (!empty($paths_array)) {
                $enabled_paths = implode(" ", array_map(function ($p) use
($baseUrl) {
                    $path = trim($p);
                    if (preg_match('/^https?:\/\//', $path)) {
                        return $path;
                    }
                    return "{$baseUrl}{$path}";
                }, $paths_array));
            } else {
                $enabled_paths = "'none'";
            }
            header("Content-Security-Policy: script-src $enabled_paths;");
        }
    }

    //analyse la requête et appelle la bonne méthode sur le bon controlleur.
    public function route(): void {
        try {
            $this->sanitize_all_input();
            //si un parametre 1, 2 ou 3 est vide (et donc non passé), le
supprimer.
            for ($i = 1; $i <= 3; ++$i) {
                if (isset($_GET["param$i"]) && $_GET["param$i"] == "") {
                    unset($_GET["param$i"]);
                }
            }
            $controller = $this->get_controller();
            if (Configuration::is_dev()) {
                $this->check_disable_js();
            }
            $this->call_action($controller);
        } catch (Exception $ex) {
```

```
            if (Configuration::is_dev())
                Tools::abort('<pre>' . $ex->getMessage() . '</pre><pre>' .
$ex->getTraceAsString() . '</pre>');
            else
                Tools::abort($ex->getMessage());
        }
    }

}
```